

Clases

Paquetes

Despliegue

Modelado con UML

Diagramas de Clases, Paquetes y Despliegue

M. en C. Beatriz Orozco Garduño

BLOQUE 1 · 45 MIN

Diagrama de Clases



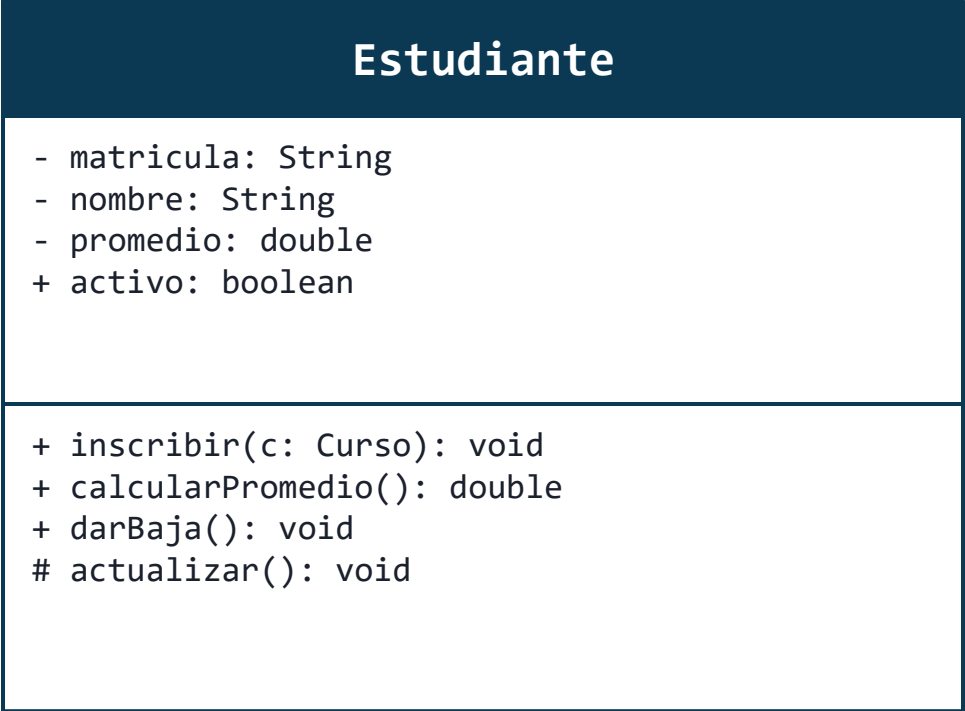
La columna vertebral del modelado estructural en UML

Diagrama de clases: notación básica

Representa la estructura estática del sistema: qué clases existen, qué características tienen y cómo se relacionan entre sí.

Cada clase se dibuja como un rectángulo dividido en tres compartimentos:

- **Nombre de la clase**
- **Atributos (estado)**
- **Operaciones (comportamiento)**



Visibilidad: + público – privado # protegido ~ paquete

Sintaxis de atributos y operaciones

Atributos

visibilidad nombre: tipo [multiplicidad] = valorInicial

Ejemplos:

- edad: int = 0
- + nombres: String [1..*]
- # saldo: double = 0.0
- ~ activo: boolean = true

Un atributo subrayado indica que es estático (de clase).

Operaciones

visibilidad nombre(param: tipo, ...): tipoRetorno

Ejemplos:

- + depositar(monto: double): void
- + buscar(id: int): Cliente
- validar(): boolean
- + listar(): List<Pedido>

Nombre de operación en itálica indica método abstracto.

Relaciones entre clases

Asociación



A — B

Conexión estructural entre clases. Puede ser uni o bidireccional.

Agregación



A ◇ — B

‘Tiene-un’ débil. El todo y la parte pueden existir por separado.

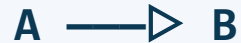
Composición



A ◆ — B

‘Tiene-un’ fuerte. Si se elimina el todo, desaparecen las partes.

Herencia



A —▷ B

‘Es-un’. La clase B es una especialización de la clase A.

Realización



A ----▷ B

Implementación de una interfaz. Línea discontinua con triángulo vacío.

Dependencia



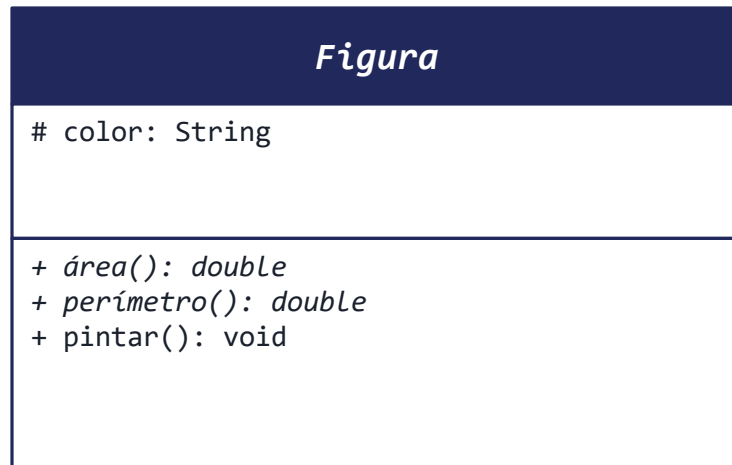
A ----> B

Uso puntual. Un cambio en B puede afectar a A.

Clases abstractas e interfaces

Clase abstracta

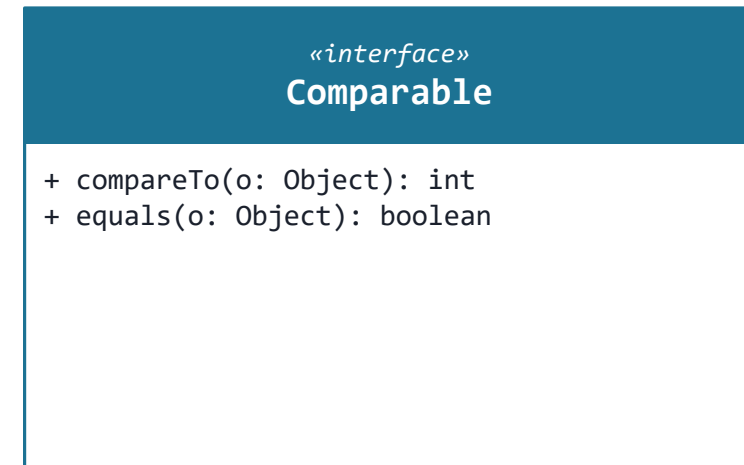
No puede instanciarse directamente. Define comportamiento común y deja operaciones abstractas para que las clases hijas las implementen. Su nombre y operaciones abstractas se escriben en *itálica*.



operaciones abstractas →

Interfaz

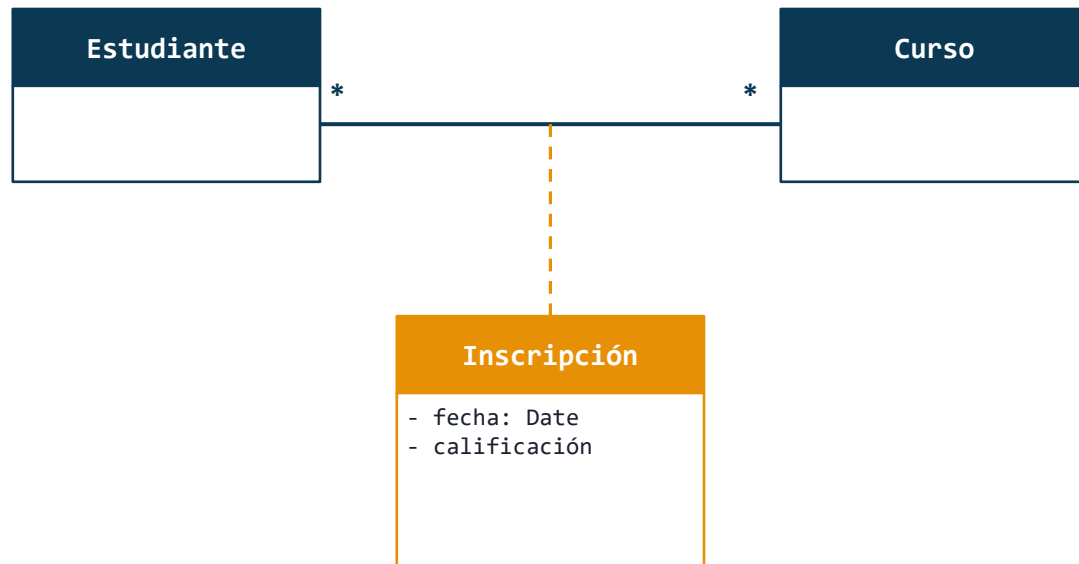
Contrato puro: solo declara operaciones sin implementación. Una clase puede implementar (realizar) varias interfaces. Se denota con el estereotipo «interface».



Clases asociación y elementos avanzados

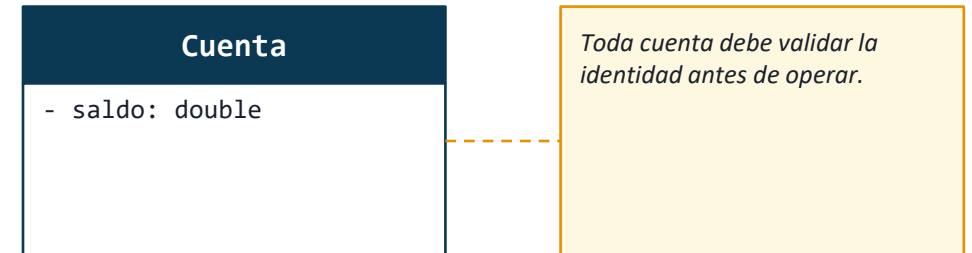
Clase asociación

Cuando una asociación tiene atributos propios, se modela como una clase conectada a la línea de asociación mediante una línea discontinua.



Notas y restricciones

Las notas aclaran detalles; las restricciones se escriben entre llaves {} y expresan reglas del negocio.



{ saldo ≥ 0 }

restricción escrita entre llaves

Asociaciones: matices importantes

Asociación reflexiva

Una clase se asocia consigo misma. Útil para jerarquías (empleado—jefe), grafos de amistad o árboles de categorías.

Empleado – supervisa → Empleado

Asociación dirigida

Se representa con una flecha. Indica que solo un extremo conoce al otro. Una sola dirección de navegación.

Pedido → Cliente

Asociación n-aria

Participan tres o más clases. Se dibuja un rombo central conectado a cada clase participante.

Equipo ◇ Jugador ◇ Temporada

Roles con nombre

Cuando una clase tiene dos asociaciones con la misma otra clase, conviene etiquetar los roles.

Persona – esposo / esposa → Persona

Cualificador

Atributo que se usa como índice para navegar desde un objeto a otro específico en una colección.

Directorio[nombre] → Archivo

Visibilidad de extremo

Puede marcarse + / – / # en los extremos de una asociación para indicar qué extremos son accesibles desde fuera.

+ productos : List<Producto>

Multiplicidad y roles en asociaciones

Notación	Significado
1	Exactamente uno
0..1	Cero o uno (opcional)
* ó 0..*	Cero o muchos
1..*	Uno o muchos (al menos uno)
n..m	Entre n y m elementos

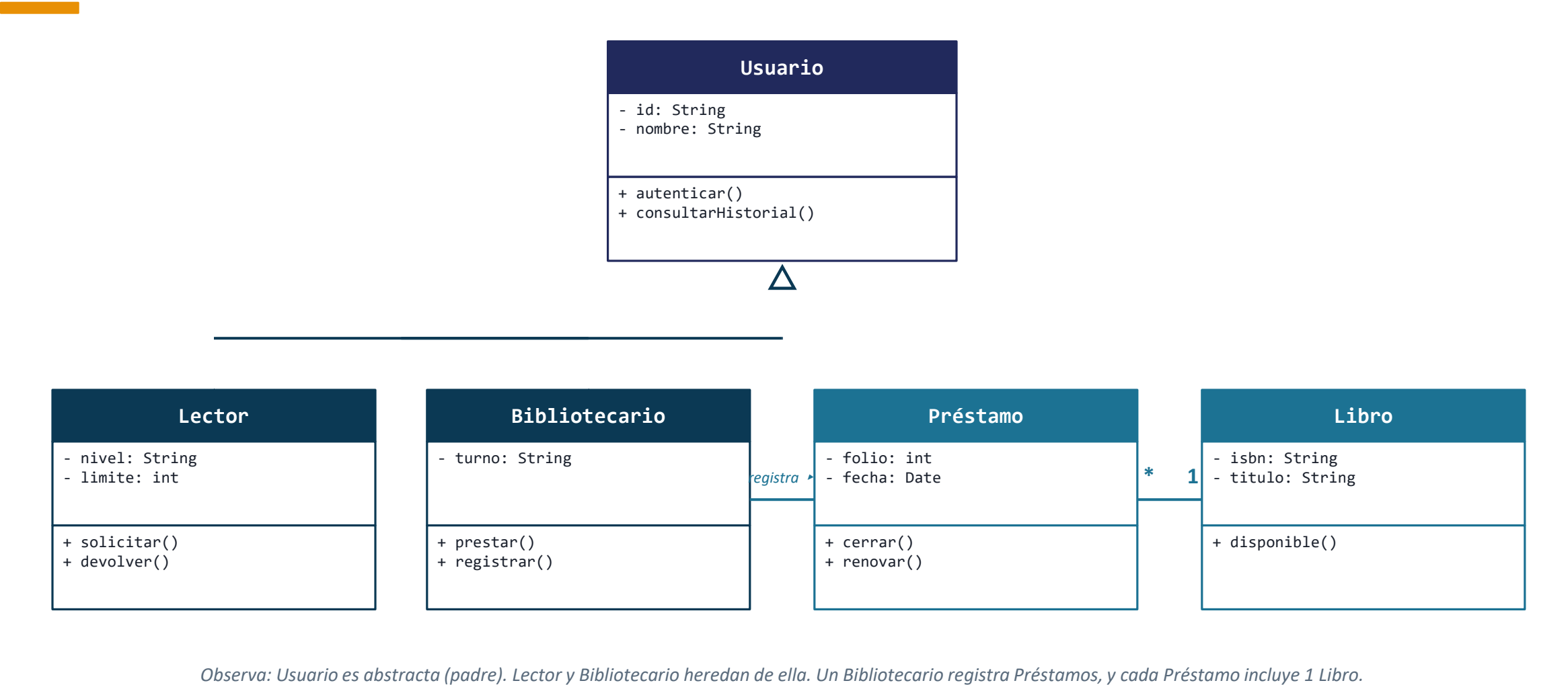
Ejemplo



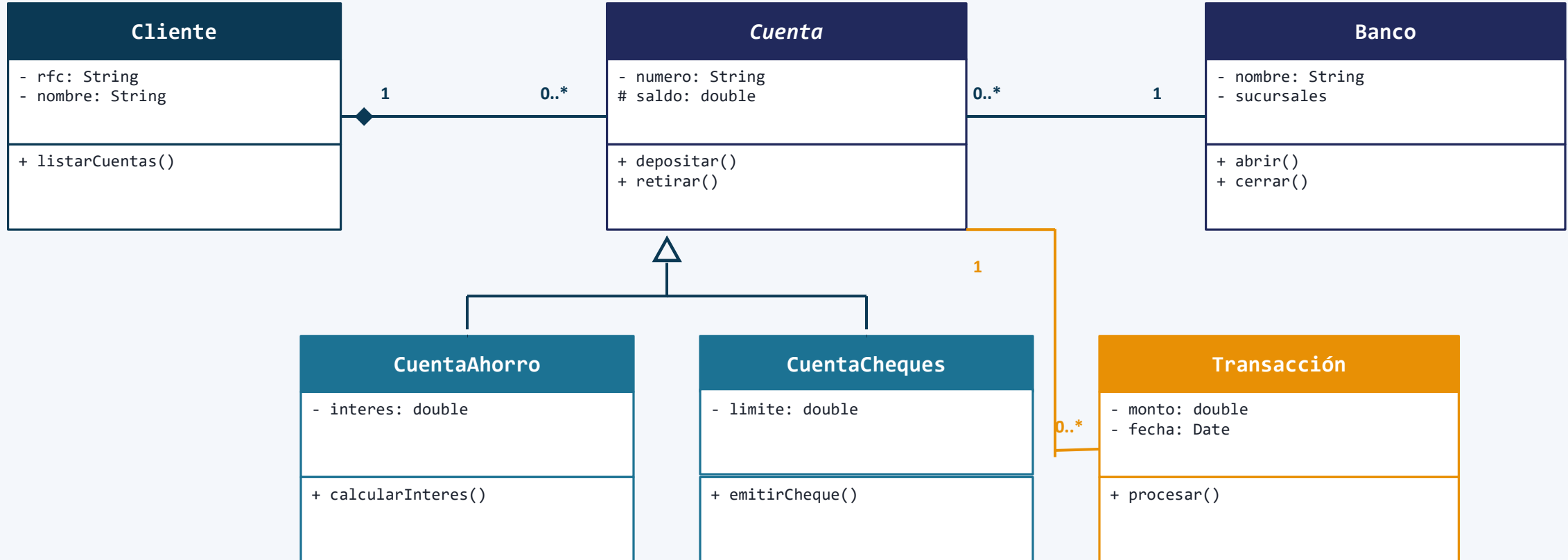
Interpretación

Un Cliente realiza cero o muchos Pedidos.
Cada Pedido pertenece a exactamente un Cliente.

Ejemplo integrador: sistema de biblioteca

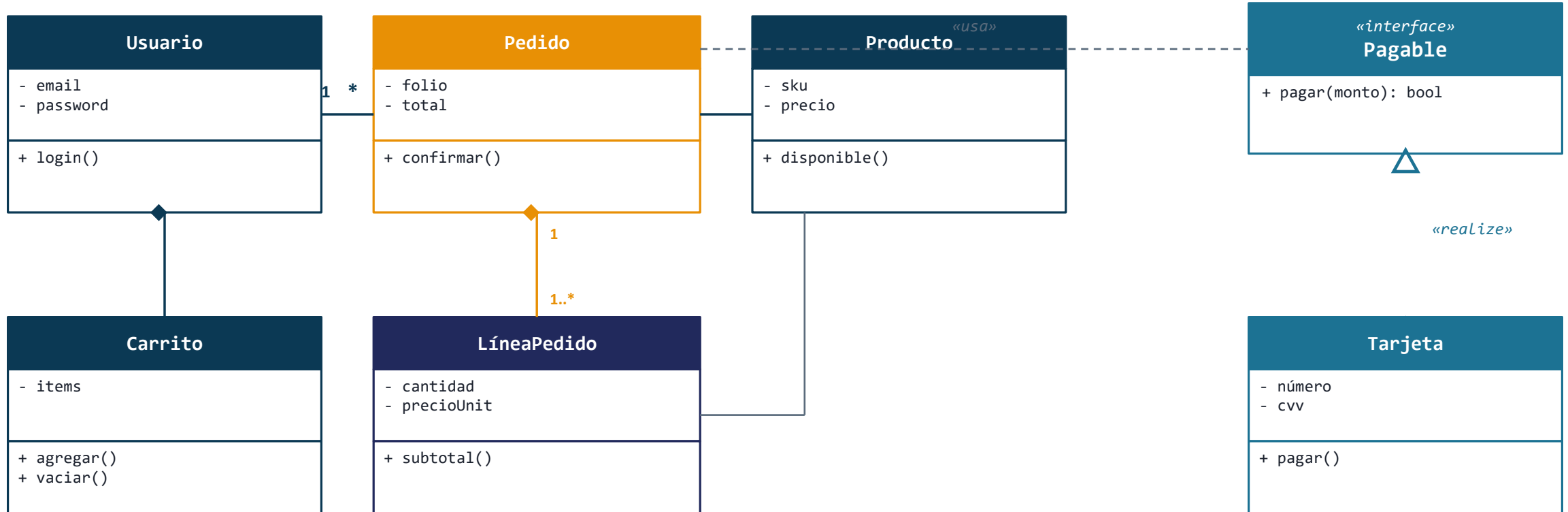


Ejemplo: sistema bancario



Lee: Cuenta es abstracta (nombre en itálica). Un Cliente posee (composición ♦) 0..* Cuentas. Cada Cuenta pertenece a 1 Banco y registra 0..* Transacciones.

Ejemplo: sistema de e-commerce (interfaces)



Lee: Pagable es una interfaz que Tarjeta realiza. Pedido la usa (dependencia) para procesar pagos sin acoplarse a un medio concreto.

Ejercicio — diagrama de clases

ENUNCIADO — 8 MINUTOS

Una clínica veterinaria gestiona dueños y sus mascotas. Cada mascota pertenece a un solo dueño, pero un dueño puede tener varias. Se registran citas médicas, atendidas por veterinarios. Cada cita produce un expediente con diagnóstico y tratamiento. Los veterinarios pueden ser generales o especialistas (con área de especialidad).

Pistas para identificar

- Sustantivos → candidatos a clases
- “pertenece a”, “tiene”, “produce” → asociaciones
- “uno/varios” → pistas de multiplicidad
- “son de tipo X o Y” → jerarquía (herencia)
- Atributos implícitos del negocio (fecha, monto, estado, etc.)

Entregables esperados

- Al menos 6 clases con atributos y operaciones
- Una relación de herencia (veterinario → tipos)
- Una composición (cita ◆— expediente)
- Multiplicidades en cada asociación
- Al menos una restricción en { llaves }

¿Qué es UML?

Unified Modeling Language

Lenguaje gráfico estandarizado por el OMG (Object Management Group) para visualizar, especificar, construir y documentar los artefactos de un sistema de software.

UML no es una metodología

Es un lenguaje de modelado. Puede usarse dentro de cualquier proceso: RUP, Scrum, Kanban, XP u otros. Su propósito es comunicar decisiones de diseño de forma clara y sin ambigüedad.

Estructurales

Clases, objetos, componentes, paquetes, despliegue, estructura compuesta

De comportamiento

Casos de uso, actividades, estados, secuencia, comunicación, tiempos

Hoy trabajaremos tres diagramas estructurales

¿Por qué modelar con UML?

01

Comunicación

Un lenguaje común entre analistas, diseñadores, desarrolladores y stakeholders.

02

Abstracción

Permite enfocarse en lo esencial del sistema, ocultando detalles de implementación.

03

Documentación

Los diagramas son artefactos vivos del proceso de ingeniería de software.

04

Análisis temprano

Detecta inconsistencias y decisiones discutibles antes de escribir código.

05

Trazabilidad

Conecta requisitos con diseño y con componentes desplegados en producción.

06

Mantenimiento

Facilita onboarding y evolución del sistema a lo largo del tiempo.

“

Todos los modelos son incorrectos; algunos son útiles.

— George E. P. Box

Buenas prácticas al modelar clases



Nombres descriptivos

Sustantivos singulares y en PascalCase: Cliente, OrdenCompra.



Nivel de detalle coherente

No mezcles clases con atributos muy detallados junto a otras vacías.



Oculto lo que no aporta

Para un diagrama de análisis, omite getters/setters triviales.



Un tema por diagrama

Si no cabe en una hoja, divide en varios diagramas por subsistema.



Multiplicidades siempre

No omitas multiplicidades: son la semántica de la asociación.



Prefiere composición sobre herencia

Usa herencia solo cuando haya verdadera sustitución de tipos.

BLOQUE 2 · 25 MIN

Diagrama de Paquetes



Organización modular del sistema y sus dependencias

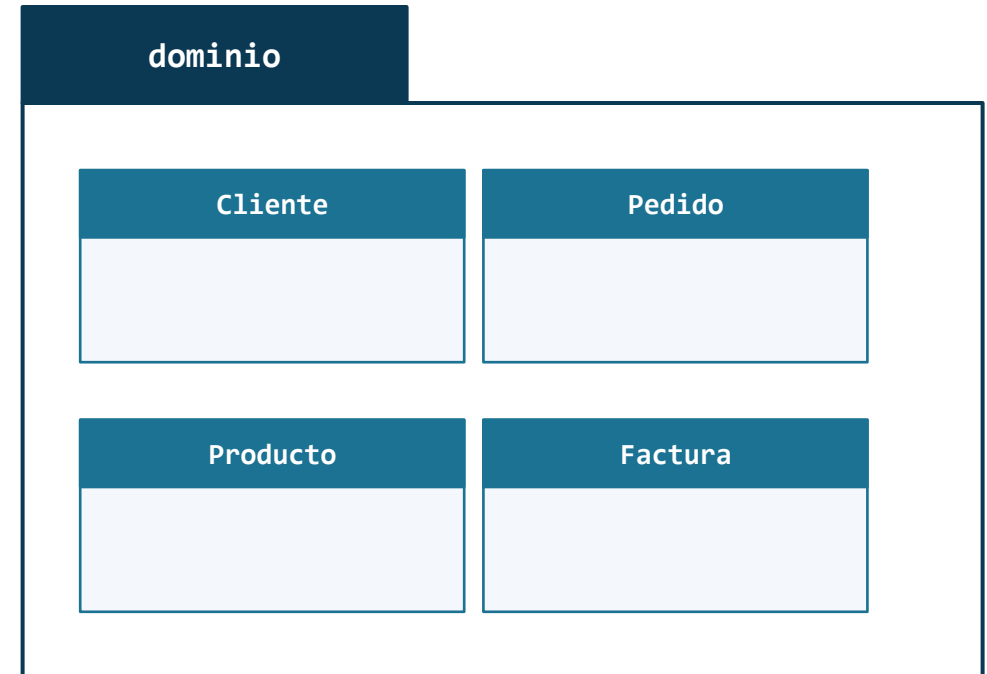
Paquetes: concepto y notación

Un paquete es un espacio de nombres que agrupa elementos de modelado relacionados.

Permite organizar clases, interfaces, otros paquetes u otros elementos UML en unidades cohesivas.

Usos típicos

- Agrupar clases por capa arquitectónica
- Reflejar la estructura de carpetas del código
- Mostrar subsistemas y sus dependencias
- Gestionar visibilidad entre módulos



Representación: una carpeta con pestaña. Dentro se dibujan los elementos que contiene.

Dependencias entre paquetes

Notación

«import»

Los elementos públicos del paquete importado quedan visibles en el que importa.

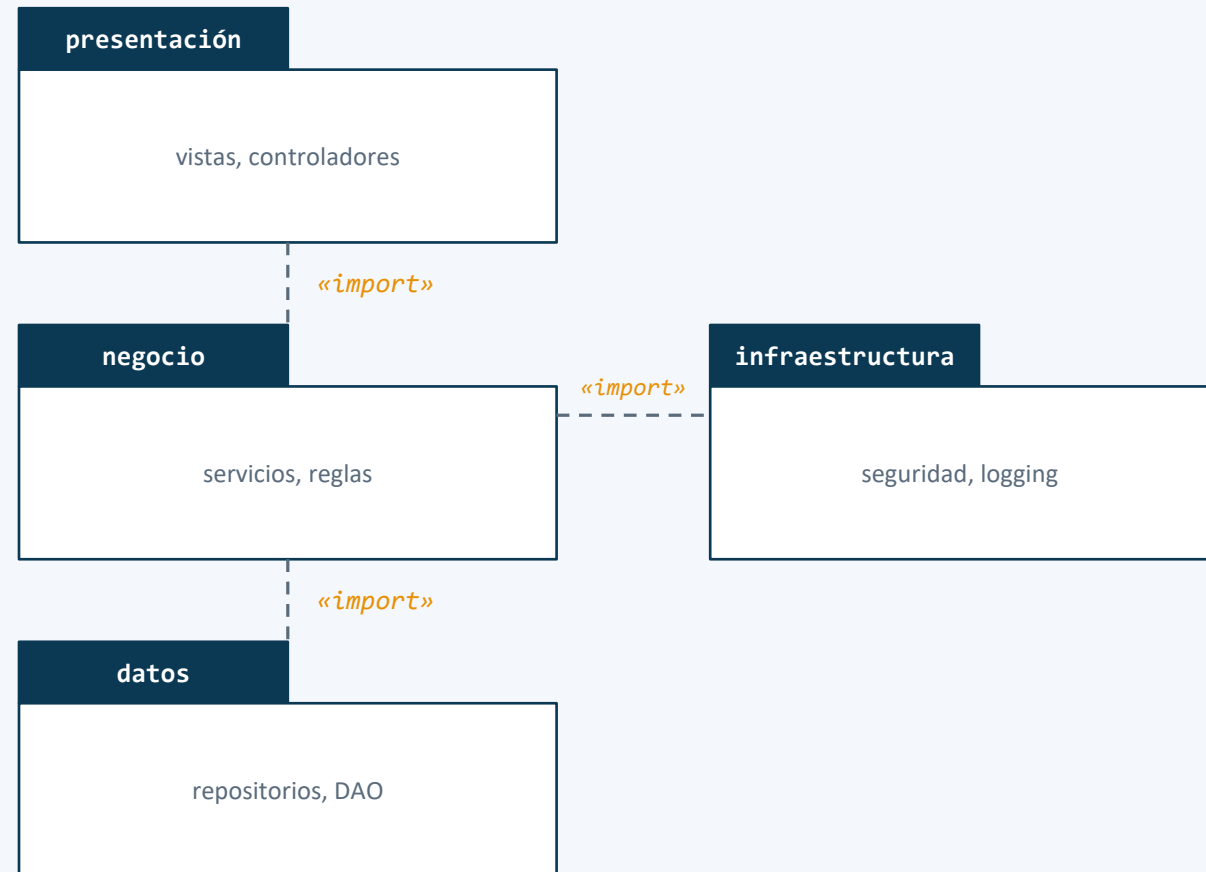
«access»

Permite usar elementos sin incorporarlos al espacio de nombres.

«merge»

Combina el contenido de dos paquetes en uno.

Evita dependencias cíclicas entre paquetes.



Arquitectura en capas con infraestructura transversal

¿Cuándo conviene un diagrama de paquetes?

Sistemas grandes

Cuando hay decenas o cientos de clases que deben comunicarse con claridad.

Arquitecturas por capas

Para visualizar separación de responsabilidades y su dirección de dependencia.

Sistemas distribuidos

Para mostrar subsistemas desplegados por separado (microservicios, módulos).

Un paquete UML suele corresponder a un directorio o namespace del código

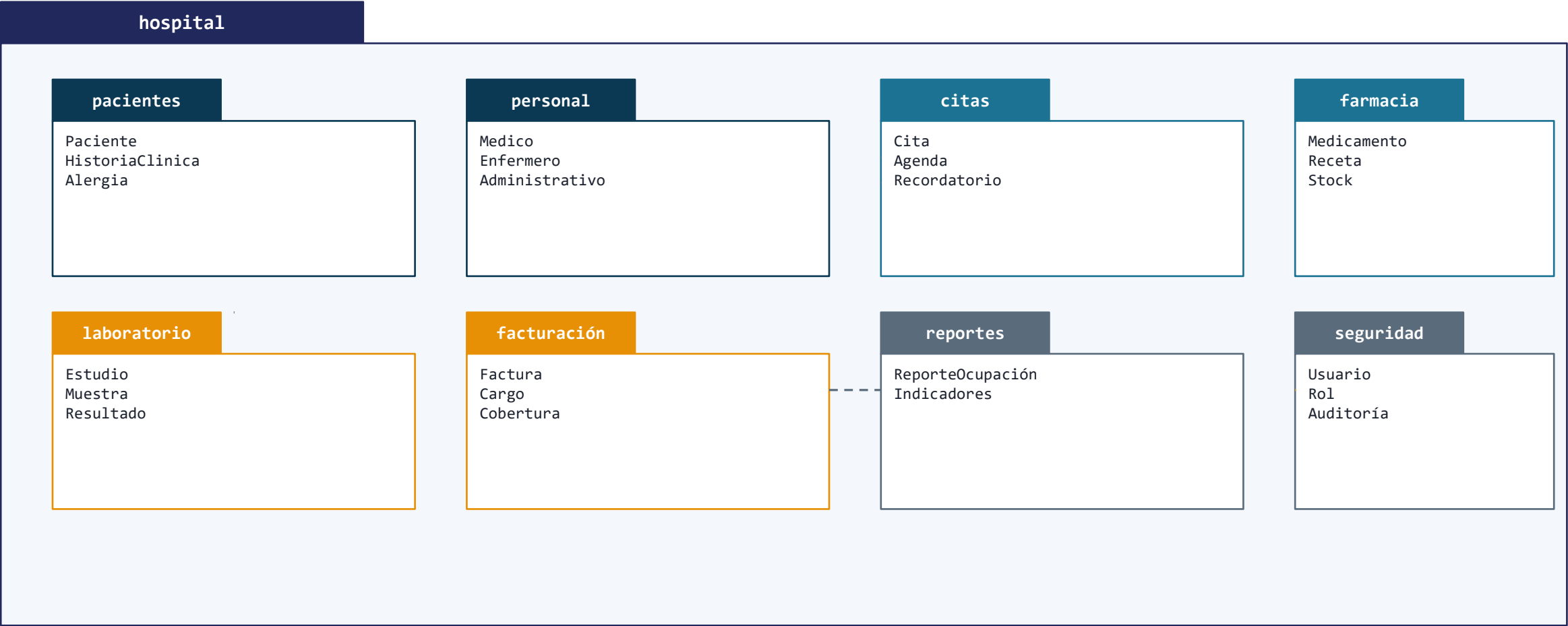
Diagrama UML

```
unam.tienda::dominio::Cliente
unam.tienda::dominio::Pedido
unam.tienda::datos::PedidoRepo
unam.tienda::ui::PedidoController
```

Estructura de código (Java/Python)

```
unam/tienda/dominio/Cliente.java
unam/tienda/dominio/Pedido.java
unam/tienda/datos/PedidoRepo.java
unam/tienda/ui/PedidoController.java
```

Ejemplo: paquetes de un sistema hospitalario



Los paquetes agrupan clases afines. Las flechas discontinuas muestran dependencias: citas usa pacientes y personal; laboratorio y facturación se apoyan en otros subsistemas; seguridad es transversal.

Ejercicio — diagrama de paquetes

ENUNCIADO — 5 MINUTOS

Dado el sistema hospitalario anterior, propón una arquitectura por capas (presentación, negocio, datos, infraestructura) donde cada paquete vertical (pacientes, citas, facturación, etc.) aparezca dividido en las capas correspondientes. Identifica las dependencias válidas y las que serían ciclos prohibidos.

	pacientes	citas	facturación
presentación	completa aquí	completa aquí	completa aquí
negocio	completa aquí	completa aquí	completa aquí
datos	completa aquí	completa aquí	completa aquí

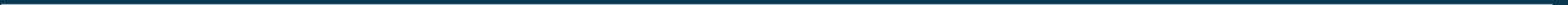
¿Puede presentación.citas depender directamente de datos.pacientes? Justifica tu respuesta.

BLOQUE 3 · 30 MIN

Diagrama de Despliegue



La arquitectura física del sistema en producción



Conceptos clave del despliegue



Nodo (node)

Recurso computacional con capacidad de procesamiento o almacenamiento. Se dibuja como un cubo 3D.

Ej.: servidor, PC, teléfono móvil, dispositivo IoT.

Artefacto

Pieza física de software: ejecutable, librería, contenedor, archivo de configuración, base de datos.

Ej.: api.jar, index.html, imagen Docker.

Asociación de comunicación

Línea entre nodos que indica protocolo o vía de comunicación entre ellos.

Ej.: <<HTTP>>, <<TCP/IP>>, <<JDBC>>.

Entorno de ejecución

Nodo especializado donde se ejecutan artefactos (JVM, navegador, contenedor).

Se representa como un nodo anidado.

Notación: nodo, artefacto y comunicación



Elementos del diagrama

Nodo: se dibuja como un cubo 3D; puede llevar un estereotipo («device», «executionEnvironment»).

Artefacto: se representa como rectángulo con el estereotipo «artifact». Se coloca dentro del nodo que lo ejecuta.

Comunicación: línea entre nodos con estereotipo del protocolo («HTTPS», «TCP/IP», «JDBC», «gRPC»).

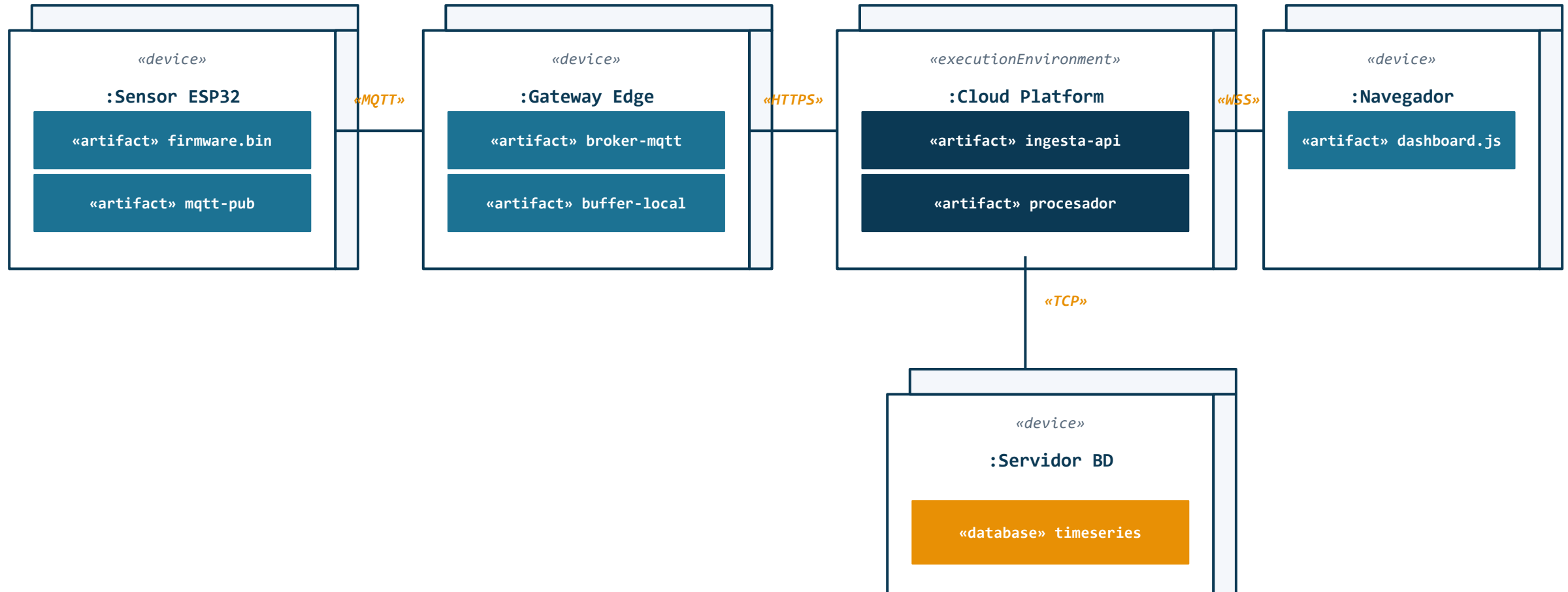
Ejemplo: aplicación web de tres capas



Lectura del diagrama

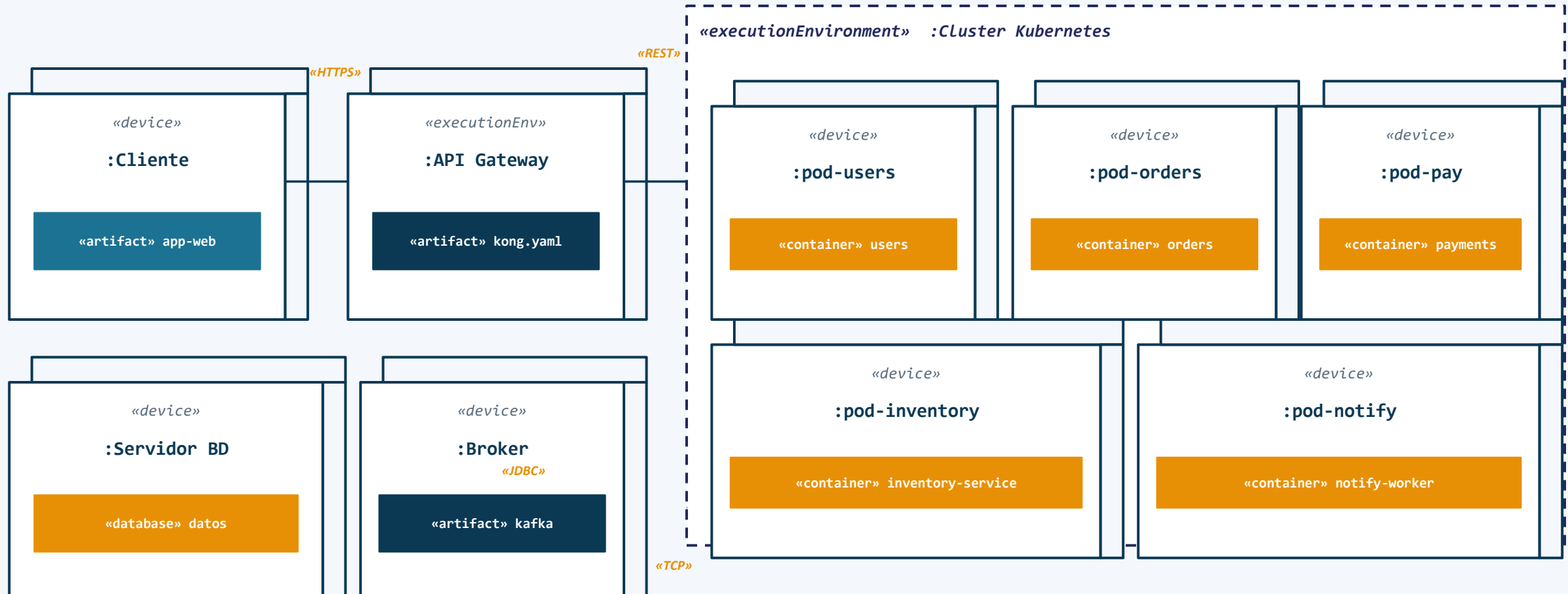
El dispositivo del usuario ejecuta dos artefactos cliente que se comunican por HTTPS con un servidor de aplicaciones. Éste aloja los servicios que, a su vez, acceden al servidor de base de datos vía JDBC con cifrado. Cada rectángulo con esquinas rectas es un artefacto; los cubos 3D son nodos físicos o entornos de ejecución.

Ejemplo: arquitectura IoT de monitoreo ambiental



Flujo: sensores publican por MQTT al gateway → gateway reenvía por HTTPS a la nube → nube persiste en BD de series temporales → navegador consume vía WebSocket seguro.

Ejemplo: microservicios con Docker y Kubernetes



Los pods se anidan dentro del entorno Kubernetes. Cada pod aloja un contenedor con un microservicio. BD y broker son nodos externos compartidos.

Ejercicio — diagrama de despliegue

ENUNCIADO — 7 MINUTOS

Una universidad quiere desplegar su sistema de calificaciones: hay una app móvil para alumnos, un portal web para docentes, un servidor de aplicaciones Java con autenticación SSO, una base de datos relacional replicada, y un servicio de notificaciones push. Todo detrás de un balanceador de carga.

1

Identifica los nodos

Dispositivos físicos y entornos de ejecución. ¿Cuántos hay? ¿Cuáles se replican?

2

Coloca los artefactos

¿Qué se ejecuta dónde? App móvil, WAR, servicios, imagen Docker, script SSO...

3

Conecta con protocolos

«HTTPS», «JDBC», «OAuth2», «FCM». Indica el estereotipo en cada línea.

Reto: ¿Qué cambiaría si el servidor de aplicaciones migrara a contenedores Kubernetes con autoescalado?

Cómo se relacionan los tres diagramas

Aspecto	Clases	Paquetes	Despliegue
Pregunta que responde	¿Qué elementos hay y cómo se relacionan?	¿Cómo está organizado el sistema?	¿Dónde se ejecuta cada parte?
Elemento principal	Clase	Paquete	Nodo y artefacto
Nivel de abstracción	Lógico fino	Lógico grueso	Físico
Momento de uso	Análisis y diseño	Arquitectura lógica	Arquitectura física
Audiencia típica	Desarrolladores	Arquitectos, devs	DevOps, arquitectos, operaciones

Los tres diagramas son complementarios: las clases viven dentro de paquetes, y los paquetes empaquetados como artefactos se despliegan en nodos.

Actividad de cierre

Reto — 10 minutos

Diseña en equipos los tres diagramas (clases, paquetes y despliegue) para un sistema de registro de calificaciones universitarias con acceso web para docentes y móvil para estudiantes.

1

Diagrama de clases

Al menos 5 clases con atributos, operaciones y relaciones con multiplicidades. Incluye una herencia y una composición.

2

Diagrama de paquetes

Organiza las clases por capas (presentación, negocio, datos) y muestra dependencias con «import».

3

Diagrama de despliegue

Identifica al menos 3 nodos, sus artefactos y los protocolos de comunicación entre ellos.